

Mash DB

A simple database implementation in Rust, built from scratch following SQLite architecture.

■ Major Feature: Dynamic Schema Support

****NEW**** - The database now supports custom table schemas with arbitrary columns! No longer limited to (id, username, email).

Examples:

```
``sql
```

```
-- Create a custom table with any columns
```

```
CREATE TABLE products (id, name, price, stock, category)
```

```
INSERT INTO products VALUES (1, 'Widget', '19.99', '100', 'Tools')
```

```
SELECT * FROM products
```

```
-- Output: (1, Widget, 19.99, 100, Tools)
```

```
-- Create another table with different schema
```

```
CREATE TABLE stores (id, name, city, opening_year, manager)
```

```
INSERT INTO stores VALUES (1, 'Downtown', 'NYC', '2020', 'John')
```

```
SELECT name, manager FROM stores
```

```
-- Output: (Downtown, John)
```

```
-- All operations work with custom columns
```

```
SELECT category, SUM(stock) FROM products GROUP BY category
```

```
SELECT name FROM products WHERE price > '15' ORDER BY name
```

```
...
```

Current Features

- ****Dynamic Schema Support**** ■ **NEW** - Create tables with any columns (no fixed structure)
- ****REPL Interface**** - Interactive command-line interface with ``db >`` prompt
- ****Meta Commands**** - ``.exit`` to quit the database, ``.save `` to save data, ``.load `` to load data
- ****Basic SQL Support****:
 - ``.INSERT INTO table VALUES (val1, val2, val3, ...)`` - Insert rows with custom columns
 - ``.SELECT * FROM table`` - Retrieve all rows with all columns

- ``SELECT col1, col2 FROM table`` - Select specific columns
- ``SELECT * FROM table WHERE column = 'value'`` - Select rows with WHERE condition
- ``SELECT DISTINCT column FROM table`` - Select unique values
- ``UPDATE table SET column = 'value' WHERE id = number`` - Update a row
- ``DELETE FROM table WHERE id = number`` - Delete a row by ID
- ``DELETE FROM table WHERE column = 'value'`` - Delete rows with WHERE condition
- **DDL Commands**:
 - ``CREATE TABLE table_name (col1, col2, col3, ...)`` - Create a new table with custom columns
 - ``DROP TABLE table_name`` - Drop an existing table (removes from registry and deletes .json file)
 - ``ALTER TABLE table_name RENAME TO new_name`` - Rename a table
- **Advanced SQL Features**:
 - **Aggregate Functions**: ``COUNT(*)``, ``COUNT(col)``, ``COUNT(DISTINCT col)``, ``SUM(col)``, ``AVG(col)``, ``MIN(col)``, ``MAX(col)``
 - **Works with custom columns**: Aggregates compute over any column
 - **ORDER BY on aggregates**: Sort grouped results by aggregate values (e.g., ``ORDER BY COUNT(*) DESC``)
 - **GROUP BY**: Group results by any columns (e.g., ``GROUP BY category, supplier``)
 - **HAVING**: Filter grouped results with conditions
 - **ORDER BY (qualified)**: Sort results ASC/DESC with table qualifiers
 - **LIMIT/OFFSET**: Paginate results
 - **LIKE Operator**: Pattern matching with ``%`` and ``_`` wildcards
 - **WHERE with NULL checks**: ``IS NULL`` and ``IS NOT NULL`` predicates
 - **Table Aliases**: Simplified references using aliases
 - **JOIN Operations**: ``INNER``, ``LEFT``, ``RIGHT`` with ``ON left.col = right.col``
 - **B-Tree Indexing** - Efficient $O(\log n)$ lookups for indexed columns
 - **In-Memory Storage** - Table stores rows in a Vec with B-Tree indexes for id
 - **Data Validation** - Column value length checks
 - **Persistence** - Save/load tables to/from JSON files
 - **Schema Registry** - Tracks column schemas for each table, persists to schemas.json
 - **Backward Compatibility** - Original (id, username, email) fixed schema still fully supported
 - **Multiple Table Support** - Different tables can have completely different schemas
 - **Comprehensive Testing** - 86+ passing tests covering all SQL features

Usage

```
```bash
```

cargo run

...

Example session:

...

db > INSERT INTO users VALUES (1, 'alice', 'alice@example.com')

Executed.

db > INSERT INTO users VALUES (2, 'bob', 'bob@example.com')

Executed.

db > INSERT INTO users VALUES (3, 'alice', 'alice2@example.com')

Executed.

db > SELECT \* FROM users

(1, alice, alice@example.com)

(2, bob, bob@example.com)

(3, alice, alice2@example.com)

Executed.

db > SELECT username, COUNT(\*), COUNT(DISTINCT email) FROM users GROUP BY username

alice, 2, 2

bob, 1, 1

Executed.

db > SELECT username FROM users GROUP BY username HAVING COUNT(\*) > 1

alice

Executed.

db > SELECT \* FROM users ORDER BY username DESC LIMIT 2

(2, bob, bob@example.com)

(3, alice, alice2@example.com)

Executed.

db > SELECT users.id, orders.id FROM users INNER JOIN orders ON users.id = orders.id

(1, 1)

(2, 2)

Executed.

db > SELECT users.username, orders.id FROM users LEFT JOIN orders ON users.id = orders.id  
ORDER BY orders.id ASC LIMIT 2

(alice, 1)

(bob, 2)

Executed.

```
db > SELECT u.username FROM users u LEFT JOIN orders o ON u.id = o.id WHERE o.id IS NULL
(charlie)
```

Executed.

```
db > SELECT username, COUNT(*) FROM orders GROUP BY username ORDER BY COUNT(*)
DESC LIMIT 2
```

```
(alice, 2)
```

```
(bob, 1)
```

Executed.

```
db > SELECT id, SUM(id) FROM orders GROUP BY id ORDER BY SUM(id) DESC
```

```
(3, 3)
```

```
(2, 2)
```

Executed.

```
db > SELECT username FROM users WHERE username LIKE 'al%'
```

```
(alice)
```

Executed.

```
db > .exit
```

```
Bye!
```

```

```

## DDL Commands - CREATE TABLE and DROP TABLE

```
```sql
```

```
-- Create a new table
```

```
db > CREATE TABLE products (id, name, price)
```

Table 'products' created with columns: id, name, price

```
-- Create another table
```

```
db > CREATE TABLE inventory (id, product_id, quantity)
```

Table 'inventory' created with columns: id, product_id, quantity

```
-- Drop a table
```

```
db > DROP TABLE inventory
```

Table 'inventory' dropped

```
-- Try to drop non-existent table (error)
```

```
db > DROP TABLE nonexistent
```

Error: Table 'nonexistent' does not exist

...

Examples - Advanced Usage

ORDER BY on Aggregate Functions

```sql

-- Count orders per user, sorted by highest count first

SELECT username, COUNT(\*) FROM orders GROUP BY username ORDER BY COUNT(\*) DESC

-- Sum amounts per user, sorted by total spending (lowest first)

SELECT username, SUM(amount) FROM orders GROUP BY username ORDER BY SUM(amount) ASC

-- Find users with most distinct order IDs

SELECT username, COUNT(DISTINCT id) FROM orders GROUP BY username ORDER BY COUNT(DISTINCT id) DESC

...

### GROUP BY with LIMIT/OFFSET on Aggregates

```sql

-- Top 3 users by order count

SELECT username, COUNT(*) FROM orders GROUP BY username ORDER BY COUNT(*) DESC LIMIT 3

-- Skip first 2 users, get next 3 by count

SELECT username, COUNT(*) FROM orders GROUP BY username ORDER BY COUNT(*) DESC LIMIT 3 OFFSET 2

...

Complex Queries Combining All Features

```sql

-- High-value orders for users with multiple orders, sorted by total

SELECT username, COUNT(\*), SUM(amount) FROM orders

WHERE amount > 100

GROUP BY username

HAVING COUNT(\*) > 1

ORDER BY SUM(amount) DESC

LIMIT 5

...

## LIKE Pattern Matching

```sql

-- Starts with 'al'

SELECT username FROM users WHERE username LIKE 'al%'

-- Ends with '@example.com'

SELECT email FROM users WHERE email LIKE '%@example.com'

-- Contains 'li'

SELECT username FROM users WHERE username LIKE '%li%'

-- Single character wildcard

SELECT username FROM users WHERE username LIKE 'b_b'

```

## Architecture

- `main.rs` - REPL loop, command parsing, and statement execution
- `table.rs` - Row and Table structures for data storage with B-Tree indexes for id, username, and email
- `column.rs` - (Reserved for future column definitions)

## TODO

- [x] Persistence (save to disk)
- [x] B-tree implementation
- [x] Multi-column indexing (username, email)
- [x] Pager for memory management
- [x] Proper SQL parser
- [x] SELECT DISTINCT
- [x] ORDER BY, LIMIT, OFFSET
- [x] GROUP BY with aggregate functions (COUNT, SUM, AVG, MIN, MAX)
- [x] HAVING clause for filtering grouped results
- [x] COUNT(DISTINCT col) for counting unique values
- [x] MIN/MAX on string columns

- [x] Multiple GROUP BY columns
- [x] Multi-table support (JOINS - INNER, LEFT, RIGHT)
- [x] ORDER BY on aggregate columns
- [x] Combined row output for JOINS
- [x] Table alias support (e.g., `users u`, `orders o`)
- [x] WHERE `IS NULL` / `IS NOT NULL`
- [x] CREATE TABLE and DROP TABLE commands
- [x] LIKE operator for pattern matching
- [x] ALTER TABLE (add/drop columns, rename) — metadata only
- [x] Subqueries — IN subquery support
- [x] Transactions — BEGIN/COMMIT/ROLLBACK (snapshot-based)

## References

- [https://cstack.github.io/db\\_tutorial/parts/part1.html](https://cstack.github.io/db_tutorial/parts/part1.html)
- <https://medium.com/@paolorechia/building-a-database-from-scratch-in-rust-part-1-6dfef2223673>